



Objective & Lecture Mapping: In previous labs, we built a Simple Reflex Agent that moved randomly or reacted only to immediate adjacent cells. Today, we transition to a **Goal-Based/Planning Agent**. You will expose the environment's state space to the agent, allowing it to use **Breadth-First Search (BFS)**, **Depth-First Search (DFS)**, and **Uniform-Cost Search (UCS)** to simulate and find paths to the food before taking a physical action.

Part 1: Practical Implementation — Building the Search Agent

Step 1.1: Exposing the World Model

Classical search algorithms require an abstract model of the environment to simulate future states. Currently, your agent is "blind" to the overall map.

1. Create a new Branch called Week_03 in your Forked Github Repo and work on the below Questions.
2. Open `visual_grid_game.py`.
3. Locate the `get_percept(self)` method.
4. Add three new keys to the dictionary to pass the global state to the agent:

```
'grid_size': (self.width, self.height)
'walls': list(self.walls)
'all_food': list(self.food_positions)
```

Step 1.2: Implementing BFS, DFS, and UCS

You will implement three distinct uninformed search strategies. The core node expansion structure is identical; only the data structure for the **Frontier** changes!

1. Open `agent.py` and create a new class called `SearchAgent` .
2. Import required structures: `from collections import deque` and `import heapq` .
3. **BFS**: Create `def bfs_search(...)` . Use a FIFO queue (`deque.popleft()`). This explores the shallowest nodes first.
4. **DFS**: Create `def dfs_search(...)` . Use a LIFO stack (`list.pop()`). This explores the deepest nodes first.
5. **UCS**: Create `def ucs_search(...)` . Use a Priority Queue (`heapq.heappop()`) ordered by the total path cost $g(n)$.
6. For all three algorithms, you must maintain a `reached` set (or visited array) to track explored states. This converts your algorithm from a *Tree Search* to a *Graph Search*, preventing infinite loops.

Step 1.3: Executing and Comparing Offline Plans

The agent must now form a complete plan and execute it step-by-step.

1. In your `SearchAgent.__init__` , add an empty list: `self.plan = []` and a config string `self.active_algo = 'BFS'` .
2. In `sense_and_act(self, percept)` , check if `self.plan` is empty.
3. If empty, find the closest food pellet from `percept['all_food']` , execute the search method matching `self.active_algo` , and store the resulting sequence of actions in `self.plan` .
4. Return the first action from the plan using `return self.plan.pop(0)` .
5. **Observation Task**: Change `self.active_algo` between 'BFS', 'DFS', and 'UCS' and run the simulation. Watch how DFS takes erratic, winding paths, while BFS and UCS take direct, optimal paths!

Part 2: Theoretical Evaluation

Based on your implementation and Lecture 04, complete the following analytical questions.

1. (Remember) You built your search logic based on the environment coordinates. What is the fundamental difference between the "State Space" and the "Search Tree"?

The state space is the complete set of all possible states that the environment can be in, together with the possible actions or transitions between those states. In `visual_grid_game.py`, each grid position can be considered a state, and the walls determine which movements between positions are possible. The search tree is the structure generated by the search algorithm while exploring the state space from a particular starting state. It represents the possible paths that the algorithm has considered. The same state can potentially appear more than once in a search tree through different paths, even though it represents only one state in the actual state space. Therefore, the state space describes the complete problem environment, while the search tree represents the portion of that space explored by a particular search process.

2. (Understand) In Step 1.2, you were instructed to use a `reached` set. In graph search theory, what specific problem does this set solve, and what would happen to your DFS agent without it?

The 'reached' set is used to prevent the algorithm from visiting and expanding the same state repeatedly. This converts the search from a tree search into a graph search and prevents unnecessary repeated exploration. This is especially important for DFS. Without the reached set, DFS could repeatedly move between connected cells and follow cycles in the grid. For example, it could move from one cell to a neighboring cell and then return to the original cell, continuing indefinitely without making useful progress toward the goal. Therefore, the reached set prevents cycles, reduces repeated work, and guarantees that already explored states are not expanded again.

3. (Analyze) Compare the visual paths generated by BFS and DFS in your simulation. Why is BFS guaranteed to be optimal in this specific grid, whereas DFS produces winding, suboptimal routes?

BFS and DFS explore the grid in different ways. BFS explores states level by level, meaning it first explores all positions that are one move away from the starting position, then all positions two moves away, and so on. Therefore, when every movement has the same cost, BFS is guaranteed to find the path containing the

smallest number of actions. In this grid, each movement has an equal cost, so BFS produces an optimal path to the food.

DFS, on the other hand, follows one branch as deeply as possible before backtracking. It does not compare all paths at the same depth before choosing a route. As a result, DFS may enter a long or winding path and reach the food through more moves than necessary. Therefore, the path produced by DFS is valid, but it is not guaranteed to be optimal.

4. (Evaluate) If we scaled `visual\_grid\_game.py` up to a massive 1000x1000 grid, BFS and UCS might crash before finding food. According to the time and space complexity formulas from the lecture, contrast the memory bottlenecks of BFS versus DFS.

For a very large grid, the major difference between BFS and DFS is their space complexity. Let b be the branching factor, d be the depth of the shallowest solution, and m be the maximum depth of the search tree.

BFS has a time complexity of approximately $O(b^d)$ and, more importantly for this question, a space complexity of approximately $O(b^d)$. BFS must store a large frontier containing all nodes at the current depth, as well as the reached states. Therefore, on a 1000×1000 grid, the amount of memory required by BFS can become extremely large before it reaches the food.

DFS has a time complexity of approximately $O(b^m)$ but a much smaller space complexity of approximately $O(bm)$ for the search frontier and path information. It mainly stores the current path and the remaining alternatives along that path rather than an entire level of the search tree. Consequently, DFS is much less likely to run out of memory on a massive grid.

Therefore, BFS is usually the bigger memory bottleneck, because it stores a very large frontier, whereas DFS is much more memory-efficient, although DFS can take considerably longer and may produce a non-optimal path.

Once Completed Get a PDF of the completed File and Push into the Week 03 brach in the Github Project

Finalize and Lock Lab 03 Documentation

